

Customer Churn Prediction and Analysis

Project Documentation

Project Overview

This project, part of the DEPI Initiative under the Ministry of Communications and Information Technology, aims to predict customer churn risk using machine learning (ML). Developed as a graduation project, it leverages a structured dataset to identify at-risk customers, enabling proactive retention strategies. The project spans data collection, exploration, feature engineering, model development, deployment, and monitoring, culminating in a Streamlit web application hosted on Hugging Face Spaces. Comprehensive documentation for all project phases is available online at <https://ahmed-m-fayad.gitbook.io/customer-churn-analysis-and-prediction>.

Business Objectives

- **Primary Goal:** Predict customer churn risk to identify at-risk customers for targeted retention campaigns.
 - **Success Metrics:** Achieve high F1-score and ROC-AUC for balanced precision and recall, ensuring reliable classification across churn risk levels (1–5).
 - **Stakeholder Needs:** Provide businesses with an intuitive tool to analyze customer data, visualize churn risks, and optimize resource allocation for retention efforts.
-

Project Scope

- **Included:** Data preprocessing, statistical analysis, feature engineering, classification model development, and deployment of an interactive web app.
 - **Excluded:** Real-time data streaming, advanced ensemble methods beyond initial exploration, and continuous model retraining in production.
-

Data Management

The dataset, sourced from the HackerEarth Machine Learning Challenge via Kaggle, contains approximately 36,000 customer records with 20 features, including demographics, behavioral metrics, and a target variable `churn_risk_score` (multi-class, levels 1–5).

Data Collection

- Source: Public GitHub repository hosting the Churn Risk Rate dataset.
 - Features: Includes age, gender, region_category, points_in_wallet, avg_time_spent, avg_transaction_value, avg_frequency_login_days, joining_date, last_visit_time, membership_category, feedback, and others.
 - Target: churn_risk_score, indicating churn risk level.
-

Data Exploration and Cleaning

- Initial Findings:
 - Invalid values: 'xxxxxxx' in referral_id, 'Error' in avg_frequency_login_days.
 - Missing values: region_category, preferred_offer_types, points_in_wallet.
 - Negative values: avg_time_spent, points_in_wallet, days_since_last_login.
 - Class imbalance: Detected in churn_risk_score, requiring balancing techniques.
- Cleaning Actions:
 - Replaced invalid entries ('Error', '?') with NaN.
 - Converted avg_frequency_login_days to float, replaced negatives with NaN.
 - Dropped irrelevant columns: customer_id, Name, security_no, referral_id.
 - Parsed and sorted date columns: joining_date, last_visit_time.
 - Removed rows with missing or invalid churn_risk_score (e.g., -1).
- Distribution Analysis:

- Used histograms, KDE plots, boxplots, and violin plots to identify skewed distributions and outliers.
 - Applied log transformation to avg_time_spent to reduce skewness.
 - **Missing Value Analysis:**
 - Visualized via bar plots, missingno matrix, and heatmap; no correlated missingness detected.
 - Planned imputation: Mean/median/mode or model-based methods.
 - **Sanity Check:**
 - Identified inconsistencies between referral_id and joined_through_referral (e.g., 'xxxxxxx' for referred users).
 - Recommended consulting data owners to clarify 'xxxxxxx' meaning, treating it as a separate category temporarily.
-

Data Splitting

- **Strategy:** 80% training, 20% testing with stratified sampling to maintain class distribution.
 - **Random Seed:** Set to 42 for reproducibility.
-

Exploratory Data Analysis

Advanced Analysis Statistical tests were conducted to identify relationships between features and churn_risk_score.

Oneway ANOVA

- **Purpose:** Assess if numeric feature means differ across churn risk scores.
- **Assumption Check:** Levene's test for homogeneity of variance.
 - Most features failed ($p \leq 0.05$), except age and days_since_last_login.
- **Results:**
 - age: $p > 0.05$, no significant difference.

- days_since_last_login: $p < 0.05$, significant differences, with post-hoc (Bonferroni) showing low risk (scores 1–2) vs. high risk (3–5).
 - Visualization: Bar plots for means, t-SNE plots confirming low vs. high-risk clusters.
-

Welch's t-Test

- Purpose: Compare behavioral/financial features between low (scores 1–2) and high (3–5) churn risk groups.
 - Features Tested: avg_time_spent, avg_transaction_value, avg_frequency_login_days, points_in_wallet.
 - Results: All features showed significant differences ($p < 0.05$), indicating strong predictive power.
 - Visualization: Boxplots showing clear separation between groups.
-

Key Insights

- days_since_last_login: Strongest predictor, supporting binary risk segmentation.
 - avg_time_spent: Lower values indicate higher churn risk.
 - avg_transaction_value: Financial engagement differentiates churn groups.
 - avg_frequency_login_days: Frequent logins linked to high churn, possibly due to service issues.
 - points_in_wallet: Higher loyalty points correlate with retention.
-

Feature Engineering

Feature engineering enhanced predictive power through new features, transformations, and encoding.

New Features

- Derived Metrics:
 - points_per_transaction: $\text{points_in_wallet} / \text{avg_transaction_value}$, reflecting loyalty per transaction.

- transaction_value_per_time_unit: $\text{avg_transaction_value} / \text{avg_time_spent}$, capturing spending efficiency.
 - Time-based Features:
 - last_visit_hour: Hour from last_visit_time.
 - last_visit_time_of_day: Categorized into Morning, Afternoon, Evening, Night.
 - last_visit_AMPM: AM/PM mapping.
 - joining_day_name: Day name from joining_date.
 - is_weekend: Flag for Saturday/Sunday joiners.
-

Encoding

- One-Hot Encoding: Applied to non-ordinal categorical columns (e.g., gender, region_category).
 - Ordinal Encoding: membership_category encoded as No Membership < Basic < Silver < Gold < Platinum < Premium.
 - Sentiment Encoding: feedback mapped to 1 (positive), 0 (neutral), -1 (negative).
-

Transformations

- Mathematical: Square root, square, and logarithmic (np.log1p) transformations for numeric features to address skewness.
 - Example: avg_transaction_value_sqrt, avg_transaction_value_square, avg_transaction_value_log.
 - Scaled Features: age, days_since_last_login, avg_time_spent, avg_transaction_value, avg_frequency_login_days, points_in_wallet, points_per_transaction, transaction_value_per_time_unit, last_visit_hour, avg_transaction_value_square.
-

Feature Combinations

- Categorical Combinations: Created compound features (e.g., region+preferred_offer_types) via a custom transformer to capture interactions.

Expected Impact

- Enhanced predictive power through domain-specific features.
 - Improved model convergence via skewness reduction.
 - Temporal and sentiment features capture behavioral trends.
 - Pipeline readiness ensures scalability.
-

Model Development

The project evaluated multiple classification models to predict churn_risk_score, selecting the best performer for deployment.

Model Selection

- Models Tested:
 - Logistic Regression: Baseline, interpretable, fast.
 - Decision Tree: Captures non-linear relationships, prone to overfitting.
 - Random Forest: Robust ensemble, reduces overfitting.
 - Gradient Boosting: High performance, requires tuning.
 - XGBoost: Optimized gradient boosting, efficient for tabular data.
 - Class Imbalance Handling: Applied SMOTE when minority class representation fell below 30%.
-

Training Methodology

- Cross-Validation: 3-fold stratified cross-validation for robust performance estimation.
- Metrics: Accuracy, precision, recall, F1-score, ROC-AUC (One-vs-Rest).
- Process:
 - Performed cross-validation to estimate generalization.
 - Trained models on the entire training set.

- Recorded training time for efficiency.
-

Preprocessing Pipeline

- Cleaning Pipeline: Handled missing values, outliers, and data cleaning.
 - Feature Engineering Pipeline: Applied encoding, scaling, and feature transformations.
-

Evaluation Results

- Logistic Regression: CV Accuracy: 0.6864, Test Accuracy: 0.6864, Precision: 0.6847, Recall: 0.6864, F1-Score: 0.6673, ROC-AUC: 0.9030, Training Time: 15.81s
 - Decision Tree: CV Accuracy: 0.6156, Test Accuracy: 0.6156, Precision: 0.6132, Recall: 0.6156, F1-Score: 0.6143, ROC-AUC: 0.7426, Training Time: 0.96s
 - Random Forest: CV Accuracy: 0.6791, Test Accuracy: 0.6791, Precision: 0.6702, Recall: 0.6791, F1-Score: 0.6496, ROC-AUC: 0.8954, Training Time: 22.26s
 - Gradient Boosting: CV Accuracy: 0.6862, Test Accuracy: 0.6862, Precision: 0.6802, Recall: 0.6862, F1-Score: 0.6665, ROC-AUC: 0.9023, Training Time: 548.37s
 - XGBoost: CV Accuracy: 0.6893, Test Accuracy: 0.6893, Precision: 0.6829, Recall: 0.6893, F1-Score: 0.6673, ROC-AUC: 0.9056, Training Time: 5.98s
-

Hyperparameter Tuning

- Method: RandomizedSearchCV with 10 parameter combinations, 3-fold cross-validation, optimizing F1-weighted score.
- Logistic Regression Grid:
 - C: [0.001, 0.01, 0.1, 1, 10, 100]
 - penalty: ['l1', 'l2', 'elasticnet', None]
 - solver: ['newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga']
- Results:
 - Best parameters: {'solver': 'saga', 'penalty': 'l1', 'C': 0.1}

- Best CV F1-score: 0.7027
 - Tuned model performance: Test Accuracy: 0.6991, Precision: 0.7015, Recall: 0.6991, F1-Score: 0.6786, ROC-AUC: 0.9058
 - Tuning time: 218.32 seconds
-

Feature Importance

- Method: Extracted coefficients from the tuned Logistic Regression model.
 - Top Features: Identified top 20 influential features (not detailed in reports).
 - Insights: Key churn drivers inform retention strategies and future data collection.
-

Final Model

- Selection: Tuned Logistic Regression chosen for high F1-score (0.6786), ROC-AUC (0.9058), interpretability, and efficiency.
 - Artifacts:
 - Model: best_churn_model_logistic_regression.pkl
 - Pipeline: complete_churn_pipeline.pkl
 - Evaluation Report: model_evaluation_report.pkl
-

Deployment and Monitoring

The final model was deployed as an interactive Streamlit web application on Hugging Face Spaces.

Application Details

- URL: <https://huggingface.co/spaces/Mr0Diablo/customer-churn-predictor>
- Purpose: Allows users to upload CSV files, validate data, predict churn, and visualize results.
- Features:
 - Data Upload: Validates CSV for required features (e.g., age, gender, points_in_wallet).

- Prediction: Uses complete_churn_pipeline.pkl for predictions and probabilities.
 - Visualizations: Plotly pie charts (churn risk distribution), histograms (numeric features), bar charts (categorical features).
 - Download: Exports prediction results as CSV.
 - Interface:
 - Dark-themed Streamlit app with wide layout, sidebar for uploads and info, and tabs for overview/detailed results.
 - Custom CSS for headers, buttons, and metric displays.
 - Functions:
 - load_model: Cached model loading with error handling.
 - verify_data: Checks for required features.
 - predict_churn: Generates predictions and probabilities.
 - display_predictions: Shows summary metrics and charts.
-

Development Environment

- Tools: Python 3.8+, Streamlit, Pandas, NumPy, Pickle, Plotly, scikit-learn.
 - Platform: Local development with VS Code, virtual environment; deployed on Hugging Face Spaces.
 - Dependence: Listed in requirements.txt with exact versions.
-

Deployment Process

- Repository: Created at Mr0Diablo/customer-churn-predictor.
- Files:
 - streamlit_app.py: Main app script.
 - complete_churn_pipeline.pkl: Pre-trained model.
 - requirements.txt: Dependency list.

- Steps:
 - Pushed files to Hugging Face Space via Git.
 - Configured Space for Streamlit framework.
 - Monitored build logs for dependency issues.
 - Validated functionality post-deployment.
-

Challenges and Solutions

- Pie Chart Display: Ensured both churn categories shown by including zero counts for missing categories.
 - Data Validation: Added robust checks with clear error messages.
 - Dark Theme: Used custom CSS to override Streamlit defaults.
 - Performance: Cached model with `@st.cache_resource` to reduce load times.
-

Testing

- Local: Tested UI, functionality, and error handling with `streamlit run app.py`.
- Data: Used sample CSVs with correct/incorrect columns.
- Visualizations: Verified Plotly chart rendering.
- Edge Cases: Tested empty files, large datasets, and missing model files.

Monitoring and Maintenance While not fully implemented, the following are recommended for production:

Monitoring Systems

- Performance Tracking: Monitor F1-score and ROC-AUC on new data to detect performance drift.
- Data Drift Detection: Track feature distributions (e.g., `avg_transaction_value`) for shifts using statistical tests (e.g., Kolmogorov-Smirnov).
- Error Logging: Log prediction errors and invalid inputs in the Streamlit app.

Retraining Schedules

- Triggers: Retrain if performance drops below F1-score of 0.65 or significant data drift is detected.
 - Frequency: Quarterly reviews of model performance and dataset updates.
-

Incident Response

- Plan: Define SLAs for app downtime and prediction errors; escalate to developers for model failures.
 - Alerts: Set up notifications for high error rates or infrastructure issues on Hugging Face Spaces.
-

Version Control

- Code: Managed via Git in the Hugging Face repository.
 - Data and Model: Use DVC for dataset versioning; store model versions with timestamps (e.g., best_churn_model_202505.pkl).
-

General Documentation Practices Code Documentation

- Comments and Docstrings: Included in streamlit_app.py and model training scripts for clarity.
 - README: Available at <https://github.com/Ahmed-M-Fayad/customer-churn-prediction>, detailing setup and milestones.
 - Standards: Followed Single Responsibility Principle for modular code (e.g., separate functions for model loading, prediction).
-

Project Documentation

- Reports: Generated for each milestone (data collection, analysis, feature engineering, model development, deployment).

- Evaluation Report: Saved as `model_evaluation_report.pkl`, detailing model metrics and methodology.
 - User Guide: Embedded in the Streamlit app's sidebar with usage tips.
-

Compliance and Ethics

- Data Privacy: Dropped sensitive columns (`customer_id`, `Name`, `security_no`) to protect user information.
 - Bias Mitigation: Addressed class imbalance with SMOTE; monitored for bias in predictions (e.g., `gender`, `region`).
 - Transparency: Used Model Cards-inspired structure in the evaluation report for clear performance reporting.
-

Reproducibility

- Environment: Specified Python 3.8+, scikit-learn, pandas, etc., in `requirements.txt`.
 - Random Seed: Set to 42 for train-test split and model training.
 - Pipeline: Saved `complete_churn_pipeline.pkl` ensures consistent preprocessing and prediction.
-

Conclusion The Customer Churn Prediction and Analysis project successfully delivered a robust Logistic Regression model (F1-score: 0.6786, ROC-AUC: 0.9058) and a user-friendly Streamlit app for churn prediction.

Key achievements include:

- Comprehensive data cleaning and feature engineering, enhancing predictive power.
- Rigorous model evaluation and tuning, ensuring high performance.
- Deployment of an interactive tool on Hugging Face Spaces, accessible to businesses.
- Detailed documentation for reproducibility and stakeholder alignment.

Future Improvements

- Modeling: Explore stacking Logistic Regression with XGBoost or neural networks.
- Features: Engineer additional features (e.g., interaction terms) and include feature importance visualizations.
- App: Add batch processing, real-time streaming, and light theme options.
- Monitoring: Implement automated drift detection and retraining pipelines.

Business Impact

- Enables proactive churn identification and targeted retention campaigns.
- Optimizes resource allocation for customer retention.
- Enhances customer lifetime value through timely interventions.

Appendix Development Environment

- Python: 3.8+
- Libraries: scikit-learn, pandas, numpy, matplotlib, seaborn, imblearn, xgboost, streamlit, plotly
- Platforms: Google Colab (model development), VS Code (app development), Hugging Face Spaces (deployment)

Execution Details

- Training Time: Approximately 20–35 minutes total, with hyperparameter tuning (approximately 218.32 seconds) as the bottleneck.
- Deployment Time: Instantaneous after Hugging Face build (approximately 5–10 minutes).

Repository

- GitHub: [Customer Churn Analysis and Prediction](#)
- Hugging Face Space: [Customer Churn Prediction Tool](#)
- GITBook Docs: [Online Documents](#)

Developed By

- Churn Project Team
- Last Updated: May 2025